

PROJECT 17: SCHOOL MANAGEMENT SYSTEM

Ngày 30 tháng 3 năm 2026

Project Objective

The objective of this project is to develop a database system to manage students, teachers, classes, subjects, and grades. The system should support daily school operations and provide useful reports for academic and administrative management.

Database and Programming Languages

- **Database Management System:** MySQL
- **Programming Language:** Python

Detailed Project Requirements

System Analysis and Requirements

- Manage student records, teacher assignments, subject offerings, class structures, and academic results.
- Facilitate efficient administration of academic data and school performance tracking.

Main Functionalities

- Student profile management (add/update student info).
- Teacher management and subject specialization tracking.
- Subject and class creation and scheduling.

- Grade entry and academic result reporting.
- Generation of reports on student performance and teacher assignments.

Database Design and Implementation

1. Data Model Design

- Create an ER diagram showing relationships among students, teachers, subjects, classes, and grades.
- Convert the ER model into relational schema including primary and foreign keys.

2. Table Structures

- Students (StudentID, StudentName, BirthDate, ClassID, Address)
- Teachers (TeacherID, TeacherName, Subject, Email)
- Subjects (SubjectID, SubjectName)
- Classes (ClassID, ClassName, TeacherID)
- Grades (GradeID, StudentID, SubjectID, Score)

3. Sample Data

- Add 510 sample rows for each table.
- Include the database schema diagram from MySQL Workbench.

Advanced Database Objects

- **Indexes:** Improve lookup speed for students, teachers, and grades.
- **Views:** Create views for class rosters, top students, and subject-wise performance.
- **Stored Procedures:** Automate student grade updates or class assignments.
- **User Defined Functions:** Calculate GPA or average scores per subject.
- **Triggers:** Auto-update class performance stats when new grades are inserted.

Database Security and Administration

- Create user roles for teachers, academic coordinators, and administrators.
- Secure student and academic records with access control policies.
- Backup strategies for academic data protection and quick recovery.
- Optimize performance for daily use and term-end reporting.

Python Application Development

- **Database Connection:** Use mysql-connector-python or SQLAlchemy.
- **Student Teacher Modules:** Create functions for data input, grade entry, and class management.
- **Reporting Module:** Generate scorecards, class performance charts, and teacher load summaries.
- **Interface:** Build a CLI or GUI interface for academic staff usage.

Deliverables

- A printed comprehensive report including database design, implementation details, and screenshots. Submit your report into LMS. First pages of your report must attach this PDF file.
- Link to publish Github source code and video presenting on Youtube
- SQL schema, sample data scripts, ER diagrams, and Python code.
- Demonstration of functionalities such as adding income, managing expenses, and generating reports.

Conclusion and Recommendations

- Summarize project outcomes and data integrity checks.
- Recommend future upgrades like attendance tracking, parent access, or mobile integration.

References

- List all academic and technical resources used in the project.

**NATIONAL ECONOMIC UNIVERSITY
FACULTY OF MATHEMATICAL ECONOMICS**



DATABASE MANAGEMENT SYSTEMS REPORT

**PROJECT 17: SCHOOL MANAGEMENT
SYSTEM**



17.pdf

Student: Nguyen Minh Thu

ID: 11245938

Class: DSEB 66B

Instructor: Tran Hung

Ha Noi, May 2026

Table of Contents

1. Introduction	7
2. System Analysis and Requirements	7
3. Database Design and Implementation	8
3.1 ER Diagram.....	8
3.2 Relational Schema	9
3.3 Sample Data	13
4. Advanced Database Objects	13
4.1 Indexes	13
4.2 Views	14
4.3 Stored Procedures.....	15
4.4 User-defined Functions	15
4.5 Triggers	15
4.6 Security and Backup	16
5. Python Application Development	17
6. Demonstration Results	22
7. Conclusion and Recommendations.....	23
8. References	24

1. Introduction

The School Management System is a database-driven application designed to support the daily academic and administrative operations of a school. The main purpose of the system is to manage student records, teachers, classes, subjects, class schedules, grades, and academic reports in an organized and reliable way.

In a traditional school environment, academic data is often stored in separate files or spreadsheets. This can lead to duplicated records, inconsistent student information, difficulty in tracking academic performance, and inefficient reporting. Therefore, this project proposes a structured database system using MySQL, together with a Python desktop application, to provide a more efficient and centralized solution.

The system allows school staff to manage student profiles, teacher assignments, subject offerings, class structures, academic results, and performance reports. It also supports role-based login so that different users can access different functions according to their responsibilities. For example, administrators can manage academic data, teachers can enter and update grades, students can view their own profiles and grades, and academic coordinators can generate reports.

The project uses MySQL as the database management system and Python with Tkinter as the application interface. MySQL is responsible for storing and enforcing data relationships through primary keys, foreign keys, constraints, views, stored procedures, functions, and triggers. Python provides a graphical user interface that allows users to interact with the database without writing SQL commands manually.

Overall, the School Management System demonstrates how database design, data integrity rules, security control, and application development can be combined to build a complete academic management solution.

2. System Analysis and Requirements

The system is designed to support the core academic management activities of a school. Based on the project requirements, the system must manage students, teachers, classes, subjects, schedules, and academic results. It must also provide useful reports for academic and administrative decision-making. The main functional requirements include:

First, the system must support student profile management. Administrators should be able to add new students, update student information, and view student details. Each student belongs to one class, and this relationship is enforced through a foreign key between the Students table and the Classes table.

Second, the system must support teacher management and subject specialization tracking. Each teacher has basic information such as teacher ID, teacher name, email, and subject specialization. The Subject field in the Teachers table represents the teacher's main specialization. In this project, this field is stored as a descriptive specialization rather than as a direct foreign key to the Subjects

table. This design keeps the system simple while still allowing the application and database triggers to check whether a teacher is assigned to a suitable subject.

Third, the system must support subject and class management. Subjects can be created and assigned to classes through the ClassSubjects table. This table is important because one class can study many subjects, and one subject can be taught in many classes. Therefore, the relationship between classes and subjects is many-to-many.

Fourth, the system must support class scheduling. A class schedule connects a class, a subject, a teacher, a day of the week, a time slot, and a room. This is stored in the ClassSchedules table. In this design, Classes.TeacherID represents the homeroom teacher of a class, while ClassSchedules.TeacherID represents the teacher who teaches a specific subject session for that class. Therefore, one class has one homeroom teacher but can also have several subject teachers.

Fifth, the system must support grade entry and academic result reporting. Teachers can enter or update grades for students. The Grades table stores student scores by subject. GPA or average score can then be calculated from grade records. If a student has not received any grade records yet, the GPA result is shown as NULL. In this case, NULL indicates that the student has not received any grade records yet, so the GPA cannot be calculated.

Sixth, the system must provide reports such as class rosters, top students, subject-wise performance, class schedules, teacher load summary, teacher assignment reports, and class performance statistics. These reports are implemented using SQL views and displayed through the Python application.

The system also includes security requirements. Access control is applied at two levels. At the database level, MySQL users, roles, and privileges are defined in security_backup.sql. At the application level, the login system redirects each user to a dashboard based on their role. This means the system combines database-level access control with application-level role-based menus.

3. Database Design and Implementation

3.1 ER Diagram

The ER diagram presents the core structure of the School Management System. The main entities include Students, Teachers, Classes, Subjects, ClassSubjects, Grades, ClassSchedules, Users, and ClassPerformanceStats.

The relationship between students and classes is one-to-many. Each student belongs to one class, while each class can have many students. This is implemented through the ClassID foreign key in the Students table.

The relationship between classes and teachers has two meanings in this system. In the Classes table, TeacherID represents the homeroom teacher of the class. Therefore, each class has one homeroom teacher. However, a class may also be taught by multiple teachers for different subjects.

This teaching assignment is represented in the ClassSchedules table, where each schedule record connects a class, subject, teacher, day, time, and room.

The relationship between classes and subjects is many-to-many. One class can study many subjects, and one subject can be assigned to many classes. This relationship is handled by the ClassSubjects table, which contains ClassID and SubjectID as a composite primary key.

The relationship between students and subjects is represented through the Grades table. Each grade record links one student to one subject and stores the student's score. A student can have many grade records, but each student-subject pair should be unique to avoid duplicate grades for the same subject.

The ClassSchedules table supports subject and class scheduling by connecting classes, subjects, teachers, days, time slots, and rooms. This table is essential for managing teaching sessions and teacher assignments.

The ClassPerformanceStats table is a supporting table used to store class average scores. It is automatically updated by database triggers when grades are inserted, updated, or deleted.

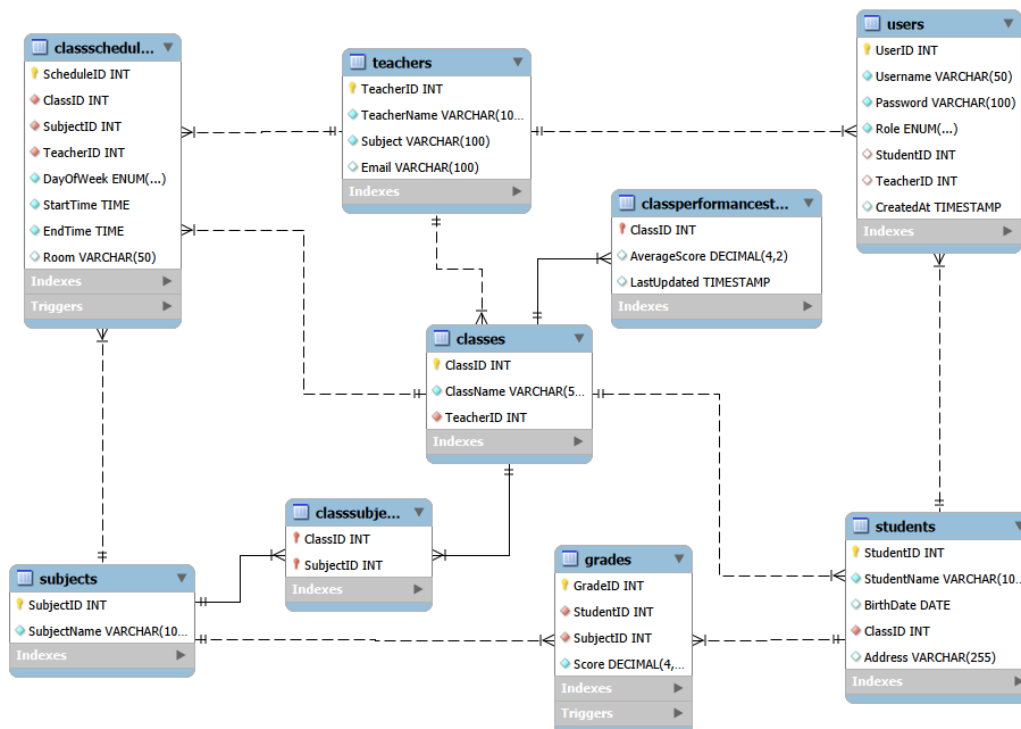


Figure 1. ER Diagram of the School Management System

3.2 Relational Schema

The database is implemented in MySQL under the schema name school_management. The main relational schema is described below.

Subjects

The Subjects table stores subject information.

```
Subjects(  
    SubjectID PRIMARY KEY,  
    SubjectName  
)
```

SubjectID uniquely identifies each subject. SubjectName stores the name of the subject, such as Database Systems, Python Programming, Mathematics, or English.

Teachers

The Teachers table stores teacher information.

```
Teachers(  
    TeacherID PRIMARY KEY,  
    TeacherName,  
    Subject,  
    Email  
)
```

TeacherID uniquely identifies each teacher. TeacherName stores the teacher's name. Subject represents the teacher's main specialization. Email stores the teacher's email address and should be unique. The Subject field in the Teachers table is not designed as a foreign key in this project. Instead, it represents the teacher's main specialization in text form. This makes the schema simpler while still allowing the application and triggers to compare the teacher specialization with the scheduled subject.

Classes

The Classes table stores class information.

```
Classes(  
    ClassID PRIMARY KEY,  
    ClassName,  
    TeacherID FOREIGN KEY REFERENCES Teachers(TeacherID)  
)
```

ClassID uniquely identifies each class. ClassName stores the name of the class, such as SE1701, DB1701, AI1701, or IS1701. TeacherID represents the homeroom teacher assigned to the class.

Students

The Students table stores student profiles.

```

Students(
    StudentID PRIMARY KEY,
    StudentName,
    BirthDate,
    ClassID FOREIGN KEY REFERENCES Classes(ClassID),
    Address
)

```

Each student belongs to one class. The foreign key ClassID ensures that a student cannot be assigned to a class that does not exist.

ClassSubjects

The ClassSubjects table defines which subjects are assigned to each class.

```

ClassSubjects(
    ClassID FOREIGN KEY REFERENCES Classes(ClassID),
    SubjectID FOREIGN KEY REFERENCES Subjects(SubjectID),
    PRIMARY KEY (ClassID, SubjectID)
)

```

This table is necessary because one class can study many subjects, and one subject can be assigned to many classes. It also supports data integrity checks for grades and schedules.

Grades

The Grades table stores student academic results.

```

Grades(
    GradeID PRIMARY KEY,
    StudentID FOREIGN KEY REFERENCES Students(StudentID),
    SubjectID FOREIGN KEY REFERENCES Subjects(SubjectID),
    Score
)

```

Each grade record belongs to one student and one subject. The Score field stores the numeric score, usually from 0 to 10. A check constraint is used to make sure that scores are within the valid range.

In addition, triggers are used to check whether the subject in a grade record belongs to the class of the student. This prevents invalid cases where a student receives a grade for a subject that his or her class does not study.

ClassSchedules

The ClassSchedules table stores teaching schedules.

```

ClassSchedules(
    ScheduleID PRIMARY KEY,
    ClassID FOREIGN KEY REFERENCES Classes(ClassID),
    SubjectID FOREIGN KEY REFERENCES Subjects(SubjectID),
    TeacherID FOREIGN KEY REFERENCES Teachers(TeacherID),
    DayOfWeek,
    StartTime,
    EndTime,
    Room
)

```

Each schedule record represents one teaching session. It connects a class, a subject, a teacher, a day, a time range, and a room. The system checks that the subject is assigned to the class before allowing a schedule to be created.

Users

The Users table is used by the Python application for login authentication.

```

Users(
    UserID PRIMARY KEY,
    Username,
    Password,
    Role,
    StudentID,
    TeacherID,
    CreatedAt
)

```

The Role field can be admin, teacher, student, or coordinator. If the user is a student, the account is linked to a StudentID. If the user is a teacher, the account is linked to a TeacherID. Admin and coordinator accounts do not need to be linked to student or teacher records.

In the demo version, passwords are stored as plain text for simplicity. In a real production system, passwords should be hashed before storage.

ClassPerformanceStats

The ClassPerformanceStats table stores average score statistics for each class.

```

ClassPerformanceStats(
    ClassID PRIMARY KEY,
    AverageScore,
    LastUpdated
)

```

This table is updated automatically by triggers whenever grade records change.

3.3 Sample Data

The project uses the Python Faker library to generate realistic sample data. The sample data includes subjects, teachers, classes, students, grades, class-subject assignments, and schedules. Using Faker makes the data more realistic because names, addresses, and other values are generated automatically. The final synchronized sample data is designed around the following logic:

- Each student belongs to exactly one class.
- Each class studies multiple subjects.
- Each subject can be assigned to multiple classes.
- Each teacher has a main subject specialization.
- Class schedules are created only for subjects that are assigned to the class.
- Student grades are generated only for subjects that belong to the student's class.

A typical dataset includes 10 subjects, 15 teachers, 8 classes, and 80 students. The system also generates multiple class-subject mappings, grade records, and class schedule records. This dataset is large enough to demonstrate GPA calculation, top student ranking, subject-wise performance, teacher workload, class rosters, and class scheduling.

The sample data is generated by a Python script and exported into `school_sample_data.sql`. This file inserts data into the database in the correct order: Subjects, Teachers, Classes, ClassSubjects, Students, Grades, ClassSchedules.

This order is important because of foreign key constraints. For example, classes must exist before students can be inserted, and subjects must exist before grade or schedule records can be created. Some results in reports may show NULL. For example, if a student has not received any grade records yet, the GPA field returns NULL. This does not indicate an error. It simply means the GPA cannot be calculated because there are no grade records for that student.

4. Advanced Database Objects

4.1 Indexes

Indexes are created to improve query performance. Since the system frequently searches students, teachers, grades, schedules, and class-subject relationships, indexes help the database retrieve records faster. The project creates indexes such as:

```
CREATE INDEX idx_students_name ON Students(StudentName);
CREATE INDEX idx_students_class ON Students(ClassID);
CREATE INDEX idx_teachers_name ON Teachers(TeacherName);
CREATE INDEX idx_teachers_subject ON Teachers(Subject);
CREATE INDEX idx_grades_student ON Grades(StudentID);
CREATE INDEX idx_grades_subject ON Grades(SubjectID);
```

```
CREATE INDEX idx_grades_score ON Grades(Score);
CREATE INDEX idx_class_subjects_class ON ClassSubjects(ClassID);
CREATE INDEX idx_class_subjects_subject ON ClassSubjects(SubjectID);
CREATE INDEX idx_schedule_class ON ClassSchedules(ClassID);
CREATE INDEX idx_schedule_subject ON ClassSchedules(SubjectID);
CREATE INDEX idx_schedule_teacher ON ClassSchedules(TeacherID);
```

These indexes are useful for daily operations and reporting. For example, `idx_grades_student` helps the system calculate a student's GPA faster, while `idx_schedule_teacher` helps generate teacher assignment reports.

If the advanced object script is executed multiple times without resetting the schema, MySQL may return a duplicate index error such as Duplicate key name. In that case, the existing indexes should be dropped before recreating them, or the database should be reset by running the schema script again.

4.2 Views

Views are used to simplify reporting queries and make the application code cleaner. Instead of writing long SQL joins repeatedly in Python, the system creates views in MySQL and then reads from them in the application. The main views include:

`vw_class_roster`

This view displays the list of students in each class. It joins the `Classes` and `Students` tables. It is useful for administrators, teachers, and coordinators to view class membership.

`vw_class_subjects`

This view displays the subjects assigned to each class. It joins `Classes`, `ClassSubjects`, and `Subjects`. This view is important because the system requires subjects to be assigned to a class before schedules and grades can be created.

`vw_top_students`

This view calculates the average score of each student and ranks students based on academic performance. If a student has no grades, the average score may be shown as `NULL`. This means the student has not received any grade records yet.

`vw_subject_performance`

This view summarizes performance by subject. It shows the number of grade records, average score, minimum score, and maximum score for each subject. This report helps academic coordinators identify strong and weak subject areas.

`vw_teacher_load_summary`

This view summarizes teacher workload. It counts the number of classes and schedule sessions assigned to each teacher. This view is based on ClassSchedules, because teaching load should be calculated from actual teaching sessions, not only from the homeroom teacher field in Classes.

vw_class_schedule

This view displays class schedules by class, subject, teacher, day, time, and room. It is used by administrators, teachers, coordinators, and students to view timetable information.

vw_teacher_assignment_report

This view displays the teaching assignments of each teacher. It includes teacher name, specialization, class name, subject name, day, time, and room. This report supports academic planning and teacher workload monitoring.

4.3 Stored Procedures

Stored procedures are used to automate database operations that may be performed repeatedly. The main stored procedure in this project is: `sp_update_student_grade`

This procedure updates a student's grade based on GradeID and a new score. It also checks whether the new score is valid. The score must be between 0 and 10. Using a stored procedure has several advantages. First, it centralizes the grade update logic inside the database. Second, it reduces repeated SQL code in the application. Third, it improves consistency because grade updates follow the same rule every time.

Example: *CALL sp_update_student_grade(1, 8.5);*

This command updates the grade record with GradeID = 1 to a new score of 8.5.

4.4 User-defined Functions

The system includes a user-defined function to calculate a student's GPA or average score. The main function is: `fn_student_gpa(p_StudentID)`

This function receives a student ID and returns the average score of that student.

Example: *SELECT fn_student_gpa(1);*

If the student has grade records, the function returns the average score rounded to two decimal places. If the student has no grade records, the function returns NULL. In this case, NULL indicates that the student has not received any grade records yet, so the GPA cannot be calculated. This function is useful for the student dashboard, where each student can view their own GPA.

4.5 Triggers

Triggers are used to enforce data integrity and automatically update supporting statistics. The system includes validation triggers and performance update triggers.

Grade validation triggers

Before inserting or updating a grade, the system checks whether the score is between 0 and 10 or the subject belongs to the class of the student. This prevents invalid academic records. For example, if a student belongs to class IS1701 and IS1701 does not study SubjectID 9, then the system should not allow a grade for SubjectID 9 to be inserted for that student.

Schedule validation triggers

Before inserting or updating a class schedule, the system checks whether the subject is assigned to the class in ClassSubjects or the teacher specialization matches the scheduled subject. This explains why a schedule cannot be created immediately after creating a new class. A class must first be assigned subjects through ClassSubjects. Only after that can schedules be created for those subjects.

Class performance triggers

The system also includes triggers that automatically update the ClassPerformanceStats table after grade records are inserted, updated, or deleted. This means class average scores remain updated without requiring the user to calculate them manually. The trigger-based design improves automation and reduces the risk of outdated performance statistics.

4.6 Security and Backup

The system applies access control at two levels.

First, at the database level, MySQL users and privileges are defined in security_backup.sql. The system defines three main access levels: teacher, academic coordinator, and administrator. Teachers can view academic records and update grades. Academic coordinators can view reports and academic data. Administrators have full database privileges.

Second, at the application level, the Python application implements role-based access control through the login system. After logging in, users are redirected to different dashboards depending on their roles. Administrators can manage student and grade records, teachers can enter grades and view academic information, students can view their own profiles and GPA, while academic coordinators can access academic reports such as top students, subject performance, teacher load summary, and class performance statistics.

The project separates responsibilities across several SQL files:

auth_setup.sql: creates application login users in the Users table

security_backup.sql: creates MySQL users, roles, and privileges

advanced_objects.sql: creates indexes, views, procedures, functions, and triggers

In the demo version, application passwords are stored as plain text for simplicity. However, in a production system, passwords should be hashed before being stored in the database. For backup, the database can be backed up using MySQL Workbench Data Export or the mysqldump command. Regular backups should be created before each academic term and before major grade updates.

A backup command can be written as:

```
mysqldump -u root -p school_management > school_management_backup.sql
```

A restore command can be written as:

```
mysql -u root -p school_management < school_management_backup.sql
```

This backup strategy helps protect academic data and supports quick recovery in case of data loss.

5. Python Application Development

The project provides a Tkinter-based desktop application connected to the MySQL database. The application allows users to interact with the database through a graphical interface instead of writing SQL commands directly.

The application uses mysql-connector-python to connect Python to MySQL. The database connection is managed through a configuration file, which stores the host, username, password, and database name. This improves maintainability because connection settings can be updated in one place.

The application includes a login system. Users enter their username and password, and the application checks the Users table. After successful login, each user is redirected to a dashboard according to their role. (Figure 2)

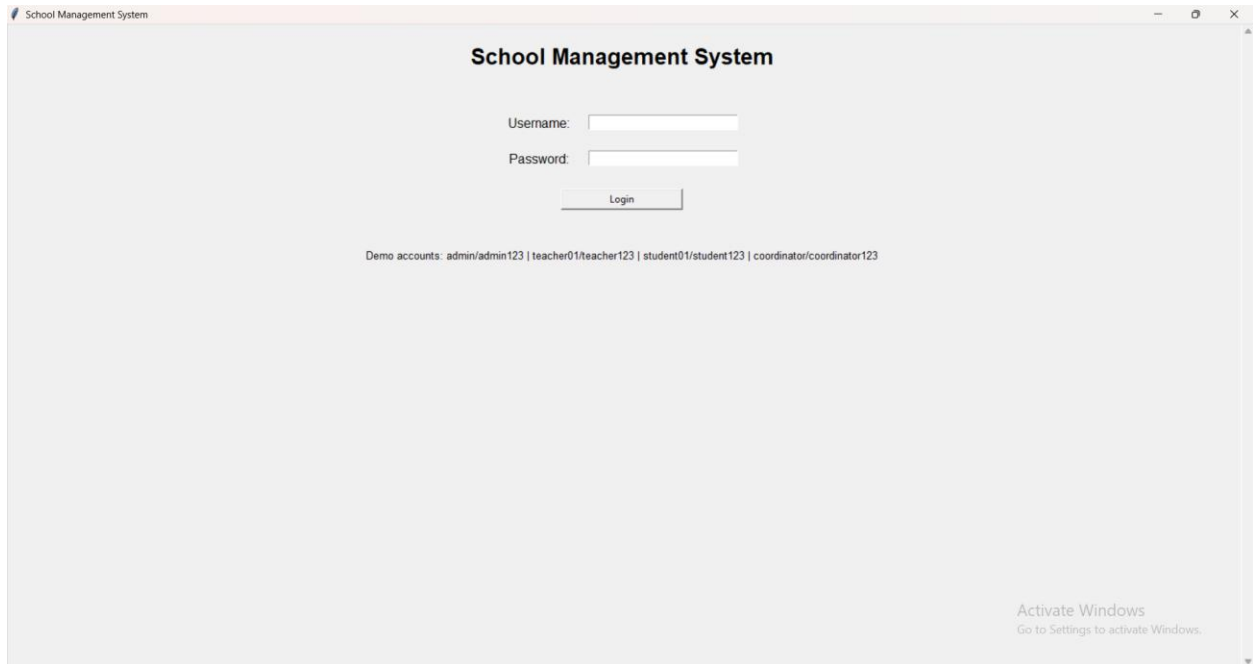


Figure 2. Login

The admin dashboard (Figure 3) provides functions such as: add new student, update student address, add new subject, create new class, assign subject to class, create class schedule, enter new grade, view class roster, view class schedule, view top students, view subject performance, view teacher load summary, view teacher assignment report, view all tables.

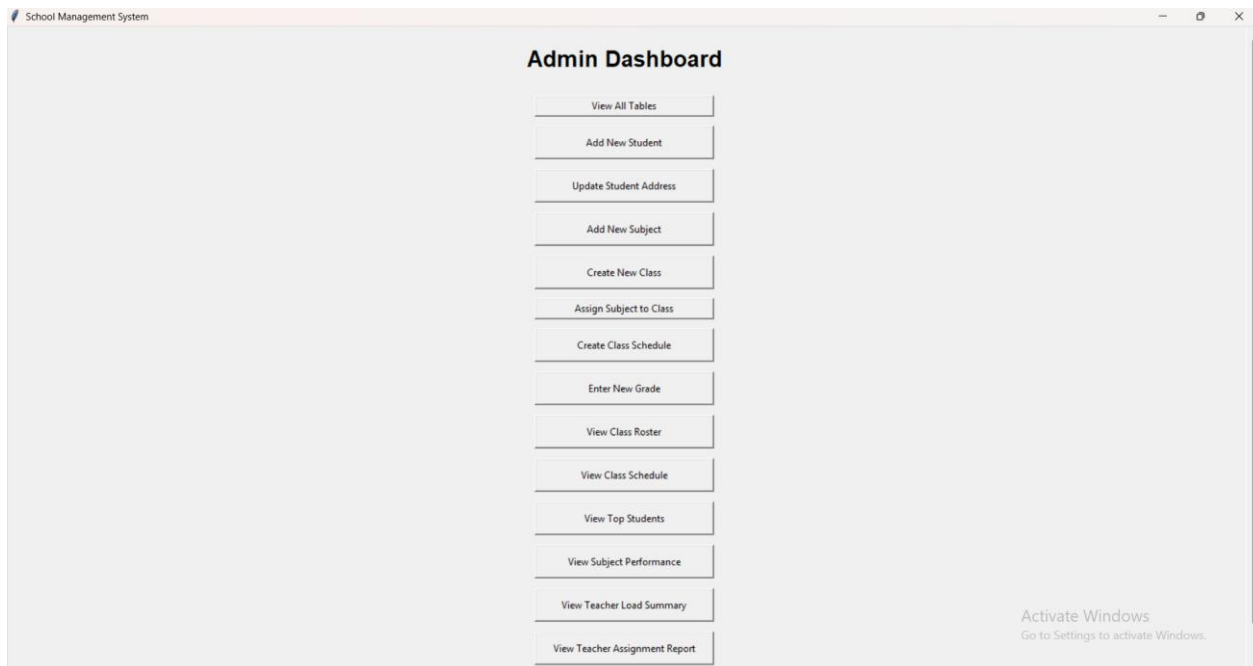


Figure 3. Admin Dashboard

School Management System

Add New Student

StudentID

StudentName

BirthDate (YYYY-MM-DD)

ClassID

Address

Activate Windows
Go to Settings to activate Windows.

Figure 4. Add New Student

School Management System

Class Roster

ClassID	ClassName	StudentID	StudentName	BirthDate	Address
7	IS1701	65	Brittney Webster	2003-10-14	6797 Nelson Thoroughway Apt. 561, Erikashire
7	IS1701	43	Bruce Collier	2007-08-08	USNV Hicks, FPO AA 54960
7	IS1701	57	Daniel Fowler	2003-07-18	29318 Brenda Via Apt. 529, West Micheleberg
7	IS1701	9	Jason Shields	2003-10-28	65133 Amanda Dam, Whitemouth, IA 10381
7	IS1701	23	John Atkinson	2003-12-25	676 Dylan Spurs, Port Michelleville, WI 56225
7	IS1701	72	John Hernandez	2003-11-21	182 John Mill Suite 889, West Nathan, LA 5121
7	IS1701	15	Lisa Archer	2008-12-03	124 Pamela Summit Apt. 449, Lake Matthew,
7	IS1701	79	Thomas Velasquez	2006-05-03	USCGC Brown, FPO AP 49516
7	IS1701	31	Tyler Reid	2007-04-10	84987 Graham Street Suite 531, Nicolebury, V
7	IS1701	80	Zachary Brooks	2003-08-08	522 Justin Village, Hortonberg, WV 92977

Activate Windows
Go to Settings to activate Windows.

Figure 5. Class Roster

School Management System

Class Schedule

ScheduleID	ClassName	SubjectName	TeacherName	DayOfWeek	StartTime	EndTime	Room
27	AI1701	Mathematics	Angie Henderson	Thursday	9:45:00	11:15:00	C302
30	AI1701	Operating Systems	Gabrielle Davis	Thursday	15:15:00	16:45:00	B202
29	AI1701	Software Engineering	Gina Moore	Friday	9:45:00	11:15:00	A102
40	BA1701	Database Systems	Jamie Arnold	Monday	8:00:00	9:30:00	C301
36	BA1701	Mathematics	Angie Henderson	Tuesday	15:15:00	16:45:00	C301
39	BA1701	Business Management	Ryan Munoz	Wednesday	8:00:00	9:30:00	B201
38	BA1701	Statistics	Connie Lawrence	Wednesday	9:45:00	11:15:00	C302
37	BA1701	English	Dylan Miller	Friday	13:30:00	15:00:00	A102
17	DB1701	Python Programming	Noah Rhodes	Monday	9:45:00	11:15:00	A102
16	DB1701	Database Systems	Allison Hill	Wednesday	15:15:00	16:45:00	D401

Back to Dashboard

Activate Windows
Go to Settings to activate Windows.

Figure 6. Class Schedule

School Management System

Teacher Assignment Report

TeacherID	TeacherName	Specialization	ClassName	SubjectName	DayOfWeek	StartTime	EndTime	Room
7	Abigail Shaffer	Computer Networks	SE1702	Computer Networks	Monday	15:15:00	16:45:00	A101
7	Abigail Shaffer	Computer Networks	SE1703	Computer Networks	Thursday	8:00:00	9:30:00	B202
1	Allison Hill	Database Systems	DB1701	Database Systems	Wednesday	15:15:00	16:45:00	D401
1	Allison Hill	Database Systems	yeah	Database Systems	Monday	1:01:02	1:01:03	D506
3	Angie Henderson	Mathematics	AI1701	Mathematics	Thursday	9:45:00	11:15:00	C302
3	Angie Henderson	Mathematics	BA1701	Mathematics	Tuesday	15:15:00	16:45:00	C301
3	Angie Henderson	Mathematics	DB1701	Mathematics	Friday	8:00:00	9:30:00	A101
3	Angie Henderson	Mathematics	DB1702	Mathematics	Thursday	9:45:00	11:15:00	D401
6	Connie Lawrence	Statistics	BA1701	Statistics	Wednesday	9:45:00	11:15:00	C302
6	Connie Lawrence	Statistics	DB1701	Statistics	Friday	15:15:00	16:45:00	C302

Back to Dashboard

Activate Windows
Go to Settings to activate Windows.

Figure 7. Teacher Assignment Report

The teacher dashboard allows teachers to enter grades and view academic information related to their work (Figure 8).

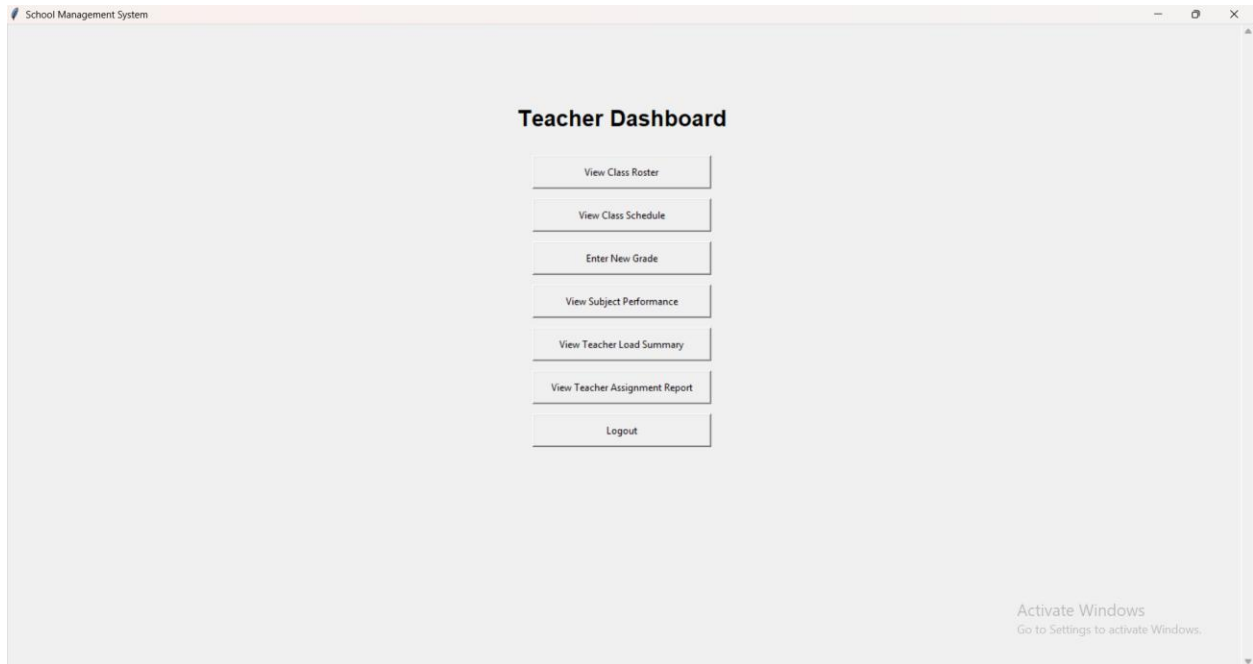


Figure 8. Teacher Dashboard

The student dashboard allows students to view their own profile, grades, and GPA (Figure 9).

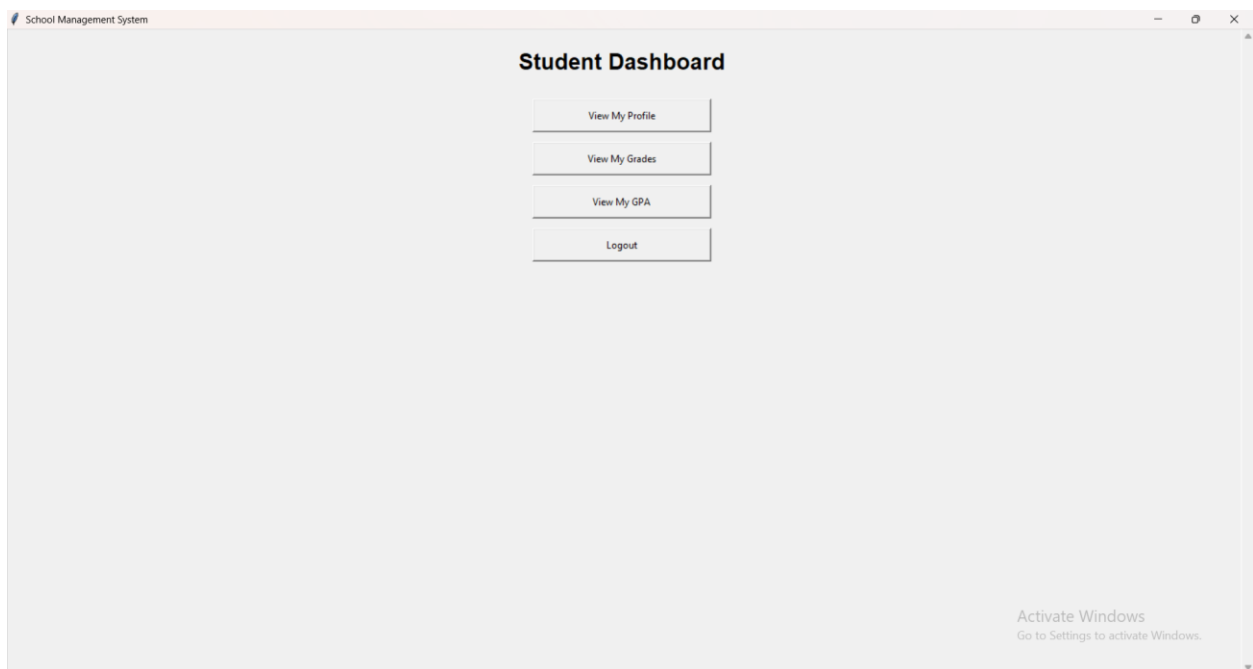


Figure 9. Student Dashboard

The academic coordinator dashboard focuses on reporting functions such as top students, subject performance, teacher workload, and class performance statistics (Figure 10).

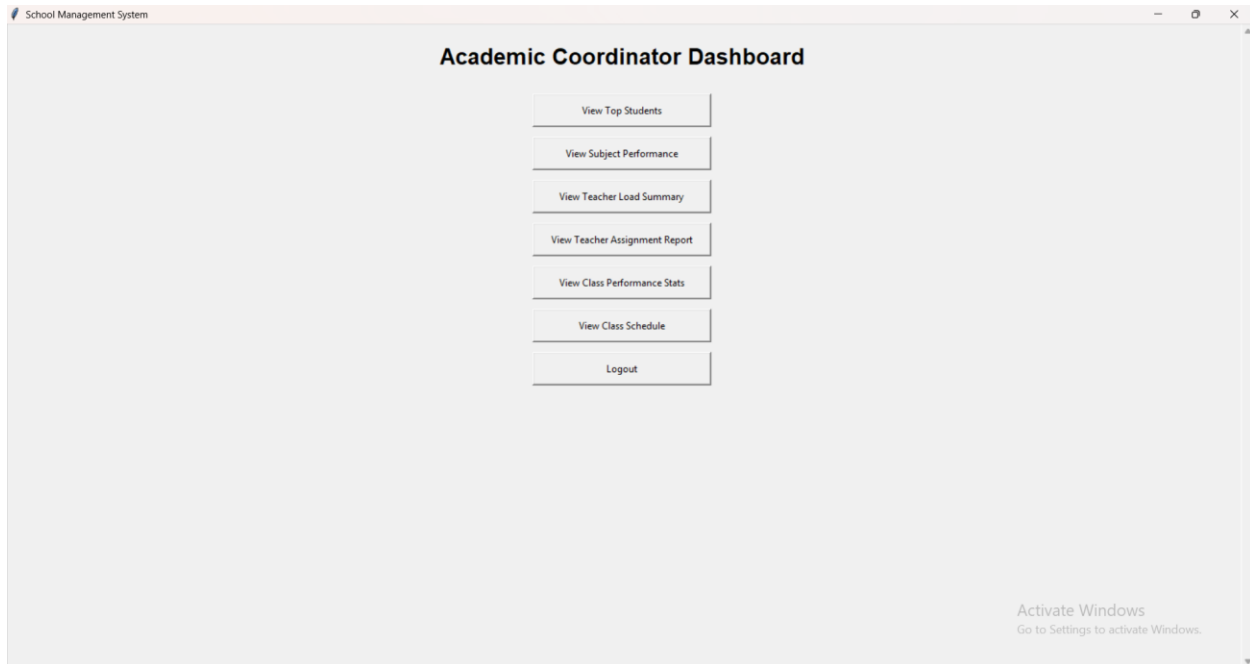


Figure 10. Academic Coordinator Dashboard

The application is packaged as an executable file using PyInstaller. Users do not need to run Python scripts manually. However, because the project uses MySQL as required, MySQL Server must be installed on the target machine. The database must also be created and initialized before the application can connect successfully.

6. Demonstration Results

The completed system demonstrates the main functions required for school management.

First, the login function works with different roles. Admin, teacher, student, and coordinator users can log in and are redirected to different dashboards. This demonstrates application-level role-based access control.

Second, the administrator can manage academic data. The admin can add new students, update student addresses, add new subjects, create new classes, assign subjects to classes, create schedules, and enter grades. These operations update the corresponding database tables.

Third, the system enforces foreign key constraints and data integrity rules. For example, a student cannot be added to a class that does not exist. A schedule cannot be created for a subject that has not been assigned to the class. A grade cannot be entered for a subject that the student's class does not study. These checks help prevent invalid data.

Fourth, the system can display academic reports. The class roster report shows students by class. The top students report ranks students by average score. The subject performance report shows the number of grades, average score, minimum score, and maximum score for each subject. The

teacher load summary shows the number of classes and teaching sessions assigned to each teacher. The teacher assignment report shows teaching schedules by teacher.

Fifth, the student dashboard allows students to view personal academic information. If a student has grade records, the GPA is calculated and displayed. If no grade records exist, the GPA is shown as NULL, meaning the GPA cannot be calculated yet.

Sixth, the system supports class scheduling. A class can be assigned several subjects through ClassSubjects, and then schedules can be created by selecting the class, subject, teacher, day, time, and room. This reflects the real structure of a school timetable.

Seventh, the system provides a view-all-tables function for administrators. This allows the administrator to inspect database tables directly from the application interface. This function is useful during testing and demonstration because it shows how application actions affect the database.

Overall, the demonstration confirms that the system can manage students, teachers, classes, subjects, schedules, grades, users, and reports in an integrated way.

7. Conclusion and Recommendations

The School Management System successfully demonstrates the design and implementation of a relational database application for academic management. The system uses MySQL to store structured school data and Python Tkinter to provide a user-friendly desktop interface.

The database design includes important academic entities such as students, teachers, classes, subjects, grades, class-subject assignments, schedules, users, and class performance statistics. Relationships are enforced through primary keys, foreign keys, composite keys, unique constraints, and check constraints. Advanced database objects such as indexes, views, stored procedures, user-defined functions, and triggers improve performance, reporting, automation, and data integrity.

The Python application allows different types of users to interact with the system through role-based dashboards. Administrators can manage academic data, teachers can enter grades, students can view their own information, and coordinators can generate reports. The system also applies access control at both the database level and the application level.

The project also includes a backup strategy using MySQL Workbench or the mysqldump command. This is important because academic data must be protected from accidental loss or system failure.

However, the system can still be improved in future versions. First, password security should be improved by hashing passwords instead of storing them as plain text. Second, the system could add attendance tracking so that teachers can record student attendance by class session. Third, parent access could be added so parents can view student performance. Fourth, the application could be upgraded to a web-based or mobile-based system for easier access. Fifth, more detailed reports and charts could be added, such as class comparison charts, grade distribution charts, and

term-based performance reports. Sixth, the application should support account creation for new students. When an administrator adds a new student, the system can automatically create a login account for that student in the Users table.

In conclusion, the project meets the main requirements of a school management database system. It provides structured data storage, academic management functions, role-based access control, useful reports, and a working Python application connected to MySQL. The system is suitable as a demonstration of database design, implementation, security, backup, and application development.

8. References

1. MySQL Documentation. *MySQL 8.0 Reference Manual*. Oracle Corporation.
2. Python Software Foundation. *Python Documentation*.
3. Tkinter Documentation. *Graphical User Interfaces with Tkinter*.
4. MySQL Connector/Python Documentation. *Connecting Python Applications to MySQL*.
5. PyInstaller Documentation. *Bundling a Python Application*.
6. Faker Documentation. *Generating Fake Data for Testing and Development*.
7. Microsoft (2022), SQL Server technical documentation, Microsoft.
8. Praat & Adamski (2005) Concepts Database Management, Thompson.
9. Microsoft SQL Server Book Online (included with Microsoft SQL Server Pack)